

Part 01

[mahmouddalii72-bot/DEPI-Tasks-Csharp-](#)

Part 02

- 1- LinkedIn article



Mahmoud Ali • You

Front Web Developer (ReactJs) ||Full stack ...

1m • 🌐



رجعنا تعاني قهوتك اللذيذة ☕

وتعالى ندردش

🧠 الذواكر اللي محدش شايفها ... Heap و Stack

🔥 "ليه بعض المتغيرات بتختفي فورًا وبعضها يعيش معاك لحد آخر البرنامج؟"

ده سر كل برنامج #C، وده اللي بيحدد حياة أي variable: مين حيعيش، ومين هيمسح، ومين محتاج Garbage Collector.

📦 مكتبك الصغير السريع - Stack

تخيل عندك مكتب صغير 📝

تحت عليه حاجة، تشتغل عليها، ولما تخلص تمسحها فورًا.

بيشتغل بالضبط كده Stack:

البيانات تتحت وتطلع بسرعة: ⚡ Fast

عمر المتغيرات قصير: Short-lived

كل → scope أو function أول ما تخرج من: Automatic cleanup

حاجة بتتمسح

ده المكان الطبيعي للـ Value Types: أرقام، struct، boolean... أي حاجة بسيطة ومباشرة.

ال Heap زي Warehouse كبير 📦

تخط فيه حاجات: كبيرة، معقدة، أو محتاجة تعيش أطول من حياة ال function اللي أنشأتها.

بيتميز بـ Heap:

أبطأ من Stack شوية 🐢

Long-lived: function البيانات ممكن تفضل موجودة بعد ما تخلص ال

مسؤول عن التنضيف Garbage Collector

ده المكان المفضل للـ Reference Types: objects, arrays, string.

الميزة هنا إن أي object ممكن يشترك فيه أكثر من reference، وال Heap بيتحمل عبء إدارة المساحة دي.

العلاقة بينهم 🔗

ال Stack غالبًا بيحمل references، وال Heap بيحمل data الحقيقية.

الورقة فيها العنوان = Stack

المخزن اللي فيه الأشياء نفسها = Heap

أي تغيير على object في Heap يظهر على أي reference في Stack

ده سر ليه بعض المتغيرات "تعيش" بعد ما تخلص ال function، وليه التغيرات أحيانًا تظهر فجأة في مكان تاني.

⚡ ليه تعرف الفرق ده مهم؟

Performance: Heap أسرع بكثير من Stack

⚡ ليه تعرف الفرق ده مهم؟

Performance: Stack أسرع بكثير من Heap

Memory management: "Memory hungry" هتفهم ليه بعض البرامج

Debugging: مش Heap في reference الغريبة سببها bugs معظم الـ Stack

🧠 الخلاصة

Stack = سريع، قصير العمر، بسيط

Heap = مسؤول Garbage Collector أبطأ، أطول عمر، كبير ومعقد، والـ عن تنظيفه

الفرق بينهم هو المفتاح لفهم أي برنامج على مستوى الـ runtime

Show translation



2- what's the difference between compiled and interpreted languages and in this way what about Csharp?

Compiled languages convert code to machine code before running.

Interpreted languages run code line by line at runtime.

C# is a hybrid language: it compiles to IL first, then JIT compiles to machine code at runtime.

3- Compare between implicit, explicit, Convert and parse casting

Implicit casting:

Automatic conversion, no data loss.

Example: `int → double`

Explicit casting:

Manual conversion, possible data loss.

Example: `(int)double`

Convert:

Uses `Convert` class, safer, handles `null`.

Example: `Convert.ToInt32("10")`

Parse:

Converts string to data type, throws error if invalid.

Example: `int.Parse("10")`